

Array vs Pointer

1. If we create a string as a ``char[]``, we can modify its characters because **its memory lives in our stack space**.
2. We can set a ``char *`` equal to another value, because **it is a reassign-able pointer**.

But we cannot set a ``char[]`` equal to another value, because it is not a pointer; **it refers to the block of memory reserved for the original array**.

3. If we pass a ``char[]`` as a parameter, set something equal to it, or perform arithmetic with it, it's **automatically converted to a ``char *``**, aka “**Decay to Pointers**”.

(Losing stack information which makes sense because the `char[]` lives in the stack where should be the caller frame; when call a new function which push into a new stack space, the callee cannot directly access to the origin stack)

```
1 void func(char *str) {
2     str[0] = 'S';
3     str++;
4     *str = 'u';
5     str = str + 3;
6     str[-2] = 'm';
7 }
8 int main(int argc, const char *argv[]) {
9     char buf[] = "Monday";
10    printf("before func: %s\n", buf);
11    func(buf);
12    printf("after func: %s\n", buf);
13    return 0;
14 }
```

Array: making space for each element in the array.

Pointer: making space for an 8 byte address.

4. If we create a new string with new characters as a ``char *``, we cannot modify its characters because its memory lives in the data segment (**read-only**).
5. Adding an offset to a C string gives us a substring that many places past the first character.

6. If we change characters in a string parameter, these changes will persist outside of the function.

```
char buf[] = "Local";  
printf("%p\n", buf); // Same address printed  
printf("%p\n", &buf); // Same address printed
```

Visual Explanation

Memory layout:

Address: 0x1000 0x1001 0x1002 0x1003 0x1004 0x1005

Content: ['L'] ['o'] ['c'] ['a'] ['l'] ['\0']

^

|

Both buf and &buf "point" here

buf: Points to the first character 'L'

&buf: Points to the entire array of 6 characters

Different Types, Different Behaviors

Here's where the difference matters:

```
char buf[] = "Local";
```

// Pointer arithmetic shows the difference:

```
printf("buf + 1: %p\n", buf + 1); // Advances by 1 char (1 byte)
```

```
printf("&buf + 1: %p\n", &buf + 1); // Advances by 1 array (6 bytes)!
```

// Example output:

// buf + 1: 0x1001 (next character)

// &buf + 1: 0x1006 (past entire array)